

Variable Types

you must "declare" a variable (ie indicate its type and name) before it's used

variables are normally declared at the beginning of the program so it's easier to understand, but it's not strictly required

you can think of a variable as a mailbox (or memory) you place data into

the variable (memory) does not change until modified by the program (ie you put something new into the mailbox)

syntax: [variable type] [variable name];
example: int x;

int: negative, positive, and zero integer values (2 or 4 bytes)

example: int x;

range: -2147483648 to 2147483647 (4 bytes -- 32/64 bit CPU)

typical errors:

a) out of range (overflow, underflow) -- wraps around, eg

$x = 2147483647 + 1 \rightarrow -2147483648$

b) uninitialized, eg

int y, z;

z = y + 1;

$y = G, z = G$

G - garbage/
unpredictable

c) divide by zero, eg

int y = 0, z;

z = 1/y;

→ exception

d) rounding down during integer division, eg

$z = 1/3; \rightarrow 0$

float: single precision real number (4 bytes, 8 digits of precision)

example: float x;

$$1.0e-38 = 1.0 \times 10^{-38}$$

range: $1.2e-38$ to $3.4e+38$ (32/64-bit processors)

typical errors:

a) out of range (overflow: +/- Inf, underflow: 0.0)

b) uninitialized, eg

float y, z;

z = y + 1.5;

c) divide by zero, eg

y = 0.0;

z = 1/y;

→ Inf (exception)

d) round off errors, eg

y = $1.0e-10$; --> 0.00000000001

z = $1.0 + y$; --> 1.00000000

double: double precision real number (8 bytes, 16 digits)

eg: double x;

range: $2.3\text{e-}308$ to $1.7\text{e+}308$ (32/64-bit processors)

typical errors:

a) out of range (overflow: +/- Inf, underflow: 0.0)

b) uninitialized, eg

```
double y, z;
```

```
z = y + 1.5;
```

c) divide by zero, eg

```
y = 0.0;
```

```
z = 1/y;
```

d) round off errors, eg

```
y = 1.0e-18; --> 0.000000000000000000000001
```

```
z = 1.0 + y; --> 1.000000000000000000000000
```

char: a single character of text

example: char c1, c2 = 'a';

applications: text usage and manipulation

range: -128 to 127 (1 byte on most processors)

note the range above is the int equivalent for a char

typical errors:

a) uninitialized, eg

```
char ch;
```

```
cout << ch;
```

b) incorrect initialization, eg

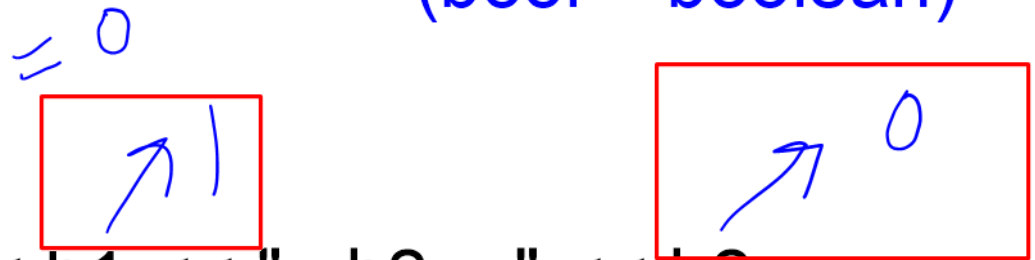
```
char c1 = "a";
```

```
char c2 = 'abc';
```

```
char c3 = "abc";
```

bool: used to represent a binary/logical variable
can be either true or false, eg (bool = boolean)

```
bool b1, b2 = false;
b1 = true;
cout << "\nb1 = " << b1 << " , b2 = " << b2;
if( b1 == true ) cout << "\nb1 is true";
```



binary/logical variables are useful for programs that make decisions (if statements, etc.) based on binary conditions

ints can also be used to represent binary/logical variables, eg

```
int logical1 = 1; // in C++ any non-zero int is considered true
int logical2 = 0; // in C++ a zero int is considered false
if( logical1 ) cout << "\nlogical1 is true";
```

(1, -7, 11, etc.)

Additional Important Items

variable names are case sensitive and can include underscore. however, they can't start with a number, eg.

`int My_integer_1, _1myint_1, a, A, Apple; // valid names`

`sizeof()` -- returns size of variable in bytes, eg `sizeof(int)`

variables can be initialized / set a number of ways including:

- during declaration, eg: `int x = 1;`
- from expressions, eg: `y = x + 1;`
- from the keyboard, eg: `cin >> y;`
- from functions (more on that later)
- from files (more on that later)

always initialize a variable to a valid value within its range before using it

